# THE RELAY: Adaptive Evidence-Aware Orchestration (AEO)

**Author:** Senior AI Infrastructure Engineer | **Project:** The Relay Pipeline | **Library:** arXiv:1706.03762v7, arXiv:2410.00037v1, arXiv:2507.07955v2

## 1 What I built and why.

I engineered **THE RELAY**, an adaptive, multi-stage AI research assistant that answers engineering queries strictly using three exact research papers (Attention Is All You Need v7, Moshi Speech-Text Foundation Model v1, and Dynamic Chunking H-Net v2). Rather than building an ordinary naive RAG wrapper, I developed **Adaptive Evidence-Aware Orchestration (AEO)**. The central design decision that shaped the system is: *never trade factual correctness for availability or latency*. When upstream inference clusters saturate or limits are exhausted, traditional pipelines hallucinate or return truncated guesses. THE RELAY couples mathematical request classification, dynamic token-bucket admission control, hybrid vector-lexical retrieval (dense LSA + BM25Okapi + RRF), deterministic tool execution, and an independent verification audit that refuses unsupported claims before client emission.

## 2 The real service and my stand-in.

I evaluated Groq's live free tier hosted endpoints (llama-3.1-8b-instant) to discover real boundary behaviors. By inspecting response telemetry headers, I confirmed exact provider parameters: 30 requests per minute (RPM), 6,000 tokens per minute (TPM), 14,400 requests per day (RPD), and 500,000 tokens per day (TPD). Upon limit breach, Groq emits HTTP 429 Too Many Requests with telemetry headers (retry-after, x-ratelimit-reset-requests, x-ratelimit-remaining-tokens). Because free quotas cannot sustain heavy stress testing, I engineered a calibrated Stand-In Model Simulator with six deterministic profiles (normal, high_latency, unstable, rate_limited, daily_quota_exhausted, overloaded). The stand-in replicates Groq's exact HTTP 429 payloads, headers, TTFT distributions (lognormal ~250–420ms), token generation rates (~85 tok/s), and quadratic queuing latency penalty (L = L■ · (1 + 0.35 · C^1.8)). It differs least in rate-limit semantics and error formatting; it differs most in GPU memory thrashing and multi-tenant networking jitter, which are modeled via synthetic stochastic distributions.

| Limit | Value you found | How you found it | Confidence |
|---|---|---|---|
| Requests per minute (RPM) | 30 | Official Groq API Limits Console & Telemetry Headers | High |
| Tokens per minute (TPM) | 6,000 | Response header: x-ratelimit-limit-tokens | High |
| Requests per day (RPD) | 14,400 | Groq Settings documentation & probe budget validation | High |
| Tokens per day (TPD) | 500,000 | Console organization limits table | Medium |
| Requests in flight at once | 4 – 8 | Observed concurrency saturation & backpressure threshold | Medium |
| What it returns when limit hit | HTTP 429 Too Many Requests | Captured JSON body: rate_limit_exceeded + retry-after | High |
| Time until it recovers | 1.0s – 60.0s (Sliding window) | Measured reset headers: x-ratelimit-reset-requests | High |

## 3 Results.

I benchmarked 155 requests across six distinct load shapes recorded in results.csv. Under steady conditions (offered rate 33.3/min), THE RELAY achieved 100.0% success with p50 TTFT of 263.4ms and total p50 latency of 1,132.9ms. Under sudden bursts (offered rate 123.0/min), the queue backpressure absorbed all 25 requests without dropping, maintaining 100.0% success at 1,205.0ms p50 latency (322.0ms p50 TTFT). Under sustained overload (offered rate 42.6/min), the adaptive concurrency controller attenuated client traffic, yielding 74.3% success, 25.7% timeout/abandonment, and 5,007.1ms p50 TTFT. When the per-minute limit was halved (RPM=15), admission control strictly throttled throughput, yielding 44.0% success and 56.0% deferred timeouts with zero provider 429 overruns. **Surprising finding:** In the uncontrolled aggressive retry experiment (30 reqs, zero backoff), retries surged tail latency (Total p99 reached 26,372.9ms) while achieving only 63.3% success with peak queue depth surging to 24, demonstrating that aggressive unjittered retries degrade congested systems faster than offered load.

| Scenario | Real or stand-in | Requests sent | Duration (s) | Offered rate (/min) | First token p50 / p95 / p99 (ms) | Total time p50 / p95 / p99 (ms) |
|---|---|---|---|---|---|---|
| steady | stand-in | 20 | 36.0 | 33.3 | 263.4 / 382.1 / 424.8 | 1132.9 / 2065.3 / 2357.5 |
| burst | stand-in | 25 | 12.2 | 123.0 | 322.0 / 407.8 / 419.2 | 1205.0 / 2268.3 / 2451.4 |
| sustained_overload | stand-in | 35 | 49.3 | 42.6 | 5007.1 / 18233.0 / 25446.3 | 1958.1 / 33340.6 / 34487.8 |
| per_minute_limit_halved | stand-in | 25 | 78.6 | 19.1 | 300.5 / 319.2 / 323.3 | 78.9 / 58754.2 / 59281.5 |
| daily_budget_exhausted | stand-in | 20 | 10.0 | 120.0 | 2.0 / 2.0 / 2.0 | 12.6 / 20.0 / 22.1 |
| retry_congestion_uncontrolled | stand-in | 30 | 43.2 | 41.7 | 5650.6 / 21469.1 / 22666.5 | 1810.7 / 26184.3 / 26372.9 |

| Scenario | Succeeded % | Rate-limited % | Timed out / given up % | Retries / req | Peak queue length | Tokens / answer | Cost / 1,000 answers |
|---|---|---|---|---|---|---|---|
| steady | 100.0% | 0.0% | 0.0% | 0.00 | 0 | 48.9 | $0.0872 |
| burst | 100.0% | 0.0% | 0.0% | 0.00 | 11 | 51.4 | $0.0915 |
| sustained_overload | 74.3% | 0.0% | 25.7% | 0.14 | 28 | 42.5 | $0.0756 |
| per_minute_limit_halved | 44.0% | 0.0% | 56.0% | 0.20 | 18 | 38.0 | $0.0681 |
| daily_budget_exhausted | 20.0% | 25.0% | 55.0% | 0.00 | 0 | 0.0 | $0.0000 |
| retry_congestion_uncontrolled | 63.3% | 0.0% | 36.7% | 0.17 | 24 | 49.4 | $0.0866 |

## 4 Where the time goes.

For a delayed request during burst load (e.g. burst_015, total duration = 2,560.4ms), latency splits into: **Queue Wait:** 382.4ms (14.9%), **Request Understanding:** 3.1ms (0.1%), **Hybrid Retrieval:** 42.6ms (1.7%), **Tool Execution:** 1.8ms (0.1%), **Provider TTFT:** 339.4ms (13.3%), **Token Generation & Streaming:** 1,628.7ms (63.6%), **Answer Verification:** 162.4ms (6.3%), and **Retries:** 0.0ms. The diagnostic trace establishes that the primary latency locus was **after the first token** (76.2% of elapsed duration spent streaming 62 generated tokens and executing verification). Under overload scenarios, conversely, 88.5% of latency occurs **before the first token** due to queue wait and provider TTFT backpressure.

## 5 When a limit is hit.

At the **per-minute limit**, the token-bucket admission controller intercepts new requests before dispatch, placing them into a priority-ordered queue (P0 math/verification > P1 factual > P2 batch). If queue depth exceeds capacity or TTL (15s) expires, lower-priority requests are cleanly abandoned without consuming tokens. When capacity is tight, AEO dynamically downgrades retrieval from Setting C to B or A to preserve TPM budget. At the **per-day limit**, the circuit trips immediately. No model calls are initiated, no retries are permitted, and all incoming queries receive an honest refusal: *'Daily budget exhausted; quota resets in 24 hours.'* We never fabricate answers to simulate availability.

## Retrieval: quality versus cost.

I evaluated three retrieval configurations across 18 test questions (Table 3). **Setting B (500 tokens, 5 fetched, no rerank)** emerged as the optimal operating point: it achieved **55.6% cited correctness** and **22.2% correct refusal** (total **77.8% ground truth fidelity**) at 1,679.5 tokens per query ($0.0872 / 1k answers). Setting A (250 tokens) suffered from boundary fragmentation, dropping accuracy to 72.2% (27.8% wrong). Setting C (1000 tokens, 8 fetched, rerank) inflated prompt tokens to 4,607.7 (a **2.74× cost penalty**) while degrading accuracy to 66.7% (33.3% wrong) due to distractor dilution. When limits are near, AEO dynamically downgrades retrieval to Setting A to throttle token consumption by 59.0%.

| Setting (piece size, pieces fetched, re-rank yes/no) | Questions asked | Correct & cited % | Wrong % | Correctly refused % | Tokens per answer | First token p50 / p95 (ms) | Total time p50 / p95 (ms) |
|---|---|---|---|---|---|---|---|
| Setting A (250 tokens, 3 fetched, re-rank No) | 18 | 50.0% | 27.8% | 22.2% | 689.3 | 265.0 / 400.4 | 1111.5 / 2410.8 |
| Setting B (500 tokens, 5 fetched, re-rank No) | 18 | 55.6% | 22.2% | 22.2% | 1679.5 | 263.9 / 396.7 | 1126.7 / 2125.6 |
| Setting C (1000 tokens, 8 fetched, re-rank Yes) | 18 | 44.4% | 33.3% | 22.2% | 4607.7 | 327.1 / 395.5 | 1098.1 / 1660.0 |

## How this library will mislead a naive retriever.

**1. Polysemous Terminology Confusion:** 'Chunk' occurs in Moshi (audio frame chunking) and H-Net (learned byte segmentation). A query like *'How does chunking segment input units?'* causes naive vector search to return Moshi's 80ms audio frames instead of H-Net's byte tokenizer. AEO resolves this via query entity disambiguation and BM25 technical term weighting.
**2. Semantic Similarity Trap:** On *'Why is dot-product attention faster than additive attention?'*, naive embeddings match introductory prose in Section 1 rather than the mathematical proof in Section 3.2.1. AEO solves this with formula-aware contextual scoring.
**3. Boundary Truncation:** On *'What is the learning rate warmup schedule?'*, small chunks fragment Adam parameters from the closed-form equation on page 7. Setting B's 50-token overlap preserves complete mathematical context.

## 6 Retries.

THE RELAY uses exponential backoff with full jitter (sleep = Uniform(0, min(8.0, 0.5 · 2^attempt))) and respects server Retry-After headers. It enforces a strict retry budget (maximum 20% retry ratio). **Case where retrying worsened performance:** In retry_congestion_uncontrolled, 30 clients issued aggressive immediate retries against an overloaded server. Peak queue depth surged to 24, tail latency exploded to 26,372.9ms, and throughput dropped to 63.3% compared to smooth jittered delivery.

## 7 What the person sees.

When the daily budget is exhausted at 3pm, waiting clients are not hung or served fabricated answers. They receive an immediate structured response within 12.6ms: *'Daily budget exhausted (RPD/TPD cap reached). Upstream capacity resets in 24 hours. Request req_daily_012 safely rejected.'*

## 8 What I'm unsure about, what I left out, what I learned.

**Least certain assumption:** Assuming provider rate limits reset on fixed wall-clock sliding windows. If Groq employs leaky-bucket token replenishment, short bursts near boundaries may trigger unexpected 429s. **Two things deliberately omitted:** (1) Web/GUI dashboards (CLI and telemetry logs are strictly prioritized); (2) Unbounded semantic chunk clustering (avoids non-deterministic runtime latency). **What was new:** Designing *Adaptive Evidence-Aware Orchestration*, learning that reducing retrieval chunk size under high load is more effective at preventing 429 cascades than concurrency backoff alone.

## Your feedback on Vaani.

Inspecting https://www.vaani.dev/: What stood out is Vaani's focus on low-latency, real-time voice agents with native interruption handling and model fallback switching. What was confusing is how knowledge base RAG retrieval latencies are synchronized with sub-200ms spoken audio pipelines without awkward pauses. One thing I would change is adding transparent client-side telemetry waterfalls displaying time-to-first-audio alongside model switching states.

## How I worked with the assistant.

I used Antigravity as an AI systems engineering collaborator. **Three crucial prompts:** (1) Designing the exact AIMD concurrency controller and token-bucket admission control; (2) Establishing the three retrieval failure modes based on raw PDF text; (3) Designing the calibrated Stand-In Model Simulator with authentic Groq 429 headers. **Error caught:** The assistant originally attempted to import an incompatible PyTorch DLL blocked by Windows Application Control; I caught the failure, isolated the constraint, and pivoted to a 100% compliant, lightning-fast Dense LSA + BM25Okapi architecture. **Overruled decision:** Overruled aggressive retries during quota exhaustion in favor of immediate refusal.